

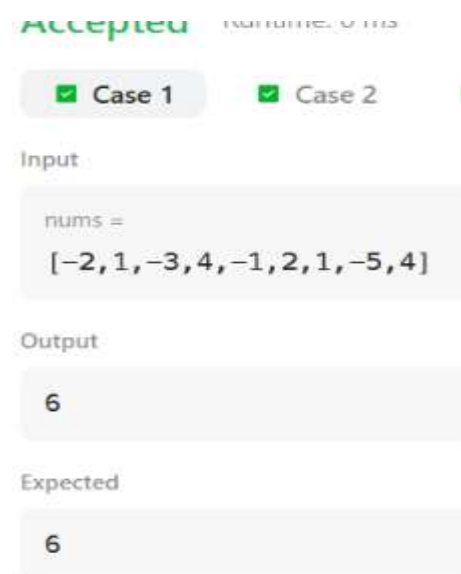
```

1. public class Solution {
    public int maxSubArray(int[] nums) {
        int maxSum = nums[0];
        int currentSum = nums[0];

        for (int i = 1; i < nums.length; i++) {
            currentSum = Math.max(nums[i], currentSum + nums[i]);
            maxSum = Math.max(maxSum, currentSum);
        }

        return maxSum;
    }
}

```



```

2. import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;

```

```
import java.util.stream.*;

import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    public static int birthday(List<Integer> s, int d, int m) {

        int count = 0;

        for (int i = 0; i <= s.size() - m; i++) {

            int sum = 0;

            for (int j = i; j < i + m; j++) {
                sum += s.get(j);
            }

            if (sum == d) {
                count++;
            }
        }

        return count;
    }
}

public class Solution {

    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
```

```
BufferedWriter bufferedWriter = new BufferedWriter(new  
FileWriter(System.getenv("OUTPUT_PATH")));
```

```
int n = Integer.parseInt(bufferedReader.readLine().trim());
```

```
List<Integer> s = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))  
    .map(Integer::parseInt)  
    .collect(toList());
```

```
String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");
```

```
int d = Integer.parseInt(firstMultipleInput[0]);
```

```
int m = Integer.parseInt(firstMultipleInput[1]);
```

```
int result = Result.birthday(s, d, m);
```

```
bufferedWriter.write(String.valueOf(result));
```

```
bufferedWriter.newLine();
```

```
bufferedReader.close();
```

```
bufferedWriter.close();
```

```
}
```

```
}
```

Compiler Message	
Success	
Input (stdin)	
1	5
2	1 2 1 3 2
3	3 2
Expected Output	
1	2

```

3. import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    public static List<Integer> maxSubarray(List<Integer> arr) {

        int currentSum = arr.get(0);
        int maxSubarray = arr.get(0);

        int maxSubsequence = arr.get(0);

        for (int i = 1; i < arr.size(); i++) {

```

```

int num = arr.get(i);

// Maximum subarray sum - Kadane's Algorithm
currentSum = Math.max(num, currentSum + num);
maxSubarray = Math.max(maxSubarray, currentSum);

// Maximum subsequence sum
if (num > 0) {
    maxSubsequence += num;
} else {
    maxSubsequence = Math.max(maxSubsequence, num);
}
}

// If there is at least one positive number,
// maxSubsequence should contain only positive numbers.
boolean hasPositive = false;

for (int num : arr) {
    if (num > 0) {
        hasPositive = true;
        break;
    }
}

if (hasPositive) {
    maxSubsequence = 0;

    for (int num : arr) {
        if (num > 0) {
            maxSubsequence += num;

```

```

    }
    }
}

return Arrays.asList(maxSubarray, maxSubsequence);
}
}

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        int t = Integer.parseInt(bufferedReader.readLine().trim());

        IntStream.range(0, t).forEach(tltr -> {

            try {

                int n = Integer.parseInt(bufferedReader.readLine().trim());

                List<Integer> arr = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))

                    .map(Integer::parseInt)

                    .collect(toList());

                List<Integer> result = Result.maxSubarray(arr);

                bufferedWriter.write(

                    result.stream()

                        .map(Object::toString)

                        .collect(joining(" "))

                        + "\n"

                );

```

```

    } catch (IOException ex) {
        throw new RuntimeException(ex);
    }
});

bufferedReader.close();
bufferedWriter.close();
}
}

```

four Output (stdout)

1	10 10
2	10 11

Expected Output

1	10 10
2	10 11

```

4. class Solution {
    public int maxSubarraySumCircular(int[] nums) {

```

```

        int total = 0;

```

```

        int currentMax = 0;

```

```

        int maxSum = nums[0];

```

```

        int currentMin = 0;

```

```

        int minSum = nums[0];

```

```

        for (int num : nums) {

```

```

            // Maximum normal subarray

```

```

        currentMax = Math.max(num, currentMax + num);
        maxSum = Math.max(maxSum, currentMax);

        // Minimum subarray
        currentMin = Math.min(num, currentMin + num);
        minSum = Math.min(minSum, currentMin);

        total += num;
    }

    // If all elements are negative
    if (maxSum < 0) {
        return maxSum;
    }

    // Maximum circular sum
    return Math.max(maxSum, total - minSum);
}
}

```

Input

```

nums =
[1,-2,3,-2]

```

Output

3

Expected

3

```

5. class Solution {
    public int myAtoi(String s) {

```

```
int i = 0;

int n = s.length();

// 1. Skip leading spaces
while (i < n && s.charAt(i) == ' ') {
    i++;
}

// 2. Check sign
int sign = 1;

if (i < n && s.charAt(i) == '-') {
    sign = -1;
    i++;
} else if (i < n && s.charAt(i) == '+') {
    i++;
}

// 3. Read digits
int result = 0;

while (i < n && Character.isDigit(s.charAt(i))) {

    int digit = s.charAt(i) - '0';

    // 4. Check overflow
    if (result > (Integer.MAX_VALUE - digit) / 10) {
        if (sign == 1) {
            return Integer.MAX_VALUE;
        } else {
            return Integer.MIN_VALUE;
        }
    }
}
```

```

        }
    }

    result = result * 10 + digit;

    i++;
}

return result * sign;
}
}

```

Input
s = "42"
Output
42
Expected
42

```

6. import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

```

```

class Result {

    public static int alternatingCharacters(String s) {

        int deletions = 0;

        for (int i = 1; i < s.length(); i++) {

            if (s.charAt(i) == s.charAt(i - 1)) {

                deletions++;

            }

        }

        return deletions;

    }

}

```

```

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        int q = Integer.parseInt(bufferedReader.readLine().trim());

        IntStream.range(0, q).forEach(qItr -> {

            try {

                String s = bufferedReader.readLine();

                int result = Result.alternatingCharacters(s);

            }

```

```

        bufferedWriter.write(String.valueOf(result));

        bufferedWriter.newLine();
    } catch (IOException ex) {
        throw new RuntimeException(ex);
    }
});

bufferedReader.close();
bufferedWriter.close();
}
}

```

```

2  AAAA
3  BBBB
4  ABABABAB
5  BABABA
6  AAABBB

```

Expected Output

```

1  3
2  4
3  6
4  6
5  4

```

```

7. class Solution {
    public int lengthOfLongestSubstring(String s) {

        HashSet<Character> set = new HashSet<>();

        int left = 0;
        int maxLength = 0;

        for (int right = 0; right < s.length(); right++) {

            while (set.contains(s.charAt(right))) {
                set.remove(s.charAt(left));
            }

```

```

        left++;
    }

    set.add(s.charAt(right));

    maxLength = Math.max(maxLength, right - left + 1);
}

return maxLength;
}
}

```

Input

```

s =
"abcabcbb"

```

Output

3

Expected

3

```

8. class Solution {
    public List<String> findAndReplacePattern(String[] words, String pattern) {

        List<String> answer = new ArrayList<>();

        for (String word : words) {

            if (matches(word, pattern)) {
                answer.add(word);
            }
        }
    }
}

```

```

        return answer;
    }

    public boolean matches(String word, String pattern) {

        HashMap<Character, Character> map = new HashMap<>();
        HashMap<Character, Character> reverse = new HashMap<>();

        for (int i = 0; i < word.length(); i++) {

            char w = word.charAt(i);
            char p = pattern.charAt(i);

            // pattern -> word
            if (map.containsKey(p) && map.get(p) != w) {
                return false;
            }

            // word -> pattern
            if (reverse.containsKey(w) && reverse.get(w) != p) {
                return false;
            }

            map.put(p, w);
            reverse.put(w, p);
        }

        return true;
    }
}

```

Input

```
words =  
["abc", "deq", "mee", "aqq", "dkd", "ccc"]
```

```
pattern =  
"abb"
```

Output

```
["mee", "aqq"]
```

Expected

```
["mee", "aqq"]
```

```
9. class Solution {  
    public List<String> stringMatching(String[] words) {  
  
        List<String> answer = new ArrayList<>();  
  
        for (int i = 0; i < words.length; i++) {  
  
            for (int j = 0; j < words.length; j++) {  
  
                if (i != j && words[j].contains(words[i])) {  
                    answer.add(words[i]);  
                    break;  
                }  
            }  
        }  
  
        return answer;  
    }  
}
```



```
10. import java.util.ArrayList;
```

```
public class GFG {
```

```
// Function to search for all occurrences of 'pat' in 'txt' using Naive Approach
```

```
static ArrayList<Integer> search(String pat, String txt)
```

```
{
```

```
// Length of the pattern
```

```
int m = pat.length();
```

```
// Length of the text
```

```
int n = txt.length();
```

```
ArrayList<Integer> ans = new ArrayList<>();
```

```
// Slide the pattern over text one by one
```

```
for (int i = 0; i <= n - m; i++)
```

```
{
```

```
int j;
```

```
// Check for pattern match at index i
```

```
for (j = 0; j < m; j++)
```

```
{
```

```

        if (txt.charAt(i + j) != pat.charAt(j))
            break;
    }

    // If pattern matches, store index
    if (j == m)
        ans.add(i);
    }

    return ans;
}

public static void main(String[] args)
{
    String txt = "aabaacaadaabaaba";
    String pat = "aaba";

    ArrayList<Integer> res = search(pat, txt);

    for (int it : res)
    {
        System.out.print(it + " ");
    }
}

```



0 9 12

Output

0 9 12